

UNIVERSITÀ DEGLI STUDI DI PADOVA

Dipartimento di Fisica e Astronomia “Galileo Galilei”

Corso di Laurea in Fisica

Tesi di Laurea

Titolo

Relatore

Prof. Paolo Umari

Laureando

Alberto Stocco

Anno Accademico xxxx/xxxx

Indice

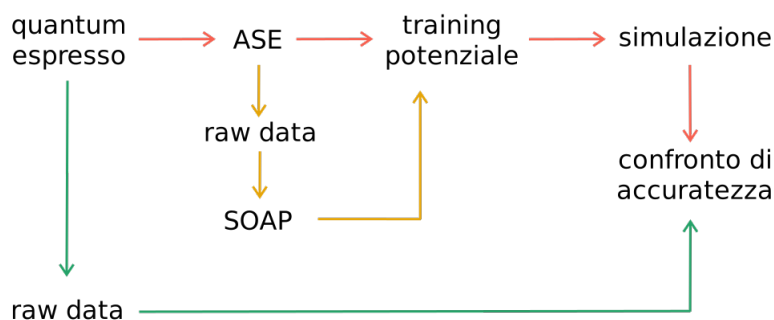
1	Struttura di lavoro	1
1.1	Quantum Espresso	1
1.2	SOAP	1
1.3	Potenziali GAP	5
1.4	Addestramento	6
1.5	Simulazione	6
2	Discussione teorica sulle librerie python utilizzate	9
2.1	ASE	9
2.1.1	Lettura dei file di Quantum Espresso	9
2.1.2	Simulazione con un potenziale ML	9
2.2	DScribe	11
2.3	Quippy	11
3	Test: dataset A	13
3.1	SOAP con Dscribe	13
3.2	Prima sessione di testing	14
3.2.1	Addestramento potenziale con Quippy	15
3.2.2	Verifica delle energie	15

Capitolo 1

Struttura di lavoro

Lo scopo che ci poniamo per questo lavoro è quello di partire da simulazioni di dinamica molecolare (MD) eseguite con Quantum Espresso (QE), simulazioni che sono costose sia dal punto di vista computazionale che temporale, ed arrivare ad un modello di machine learning (ML) che sia in grado di svolgere una simulazione il più possibile accurata e con tempi di esecuzione sensibilmente minori.

Il processo che andiamo a seguire è descritto in modo schematico qui sotto:



Schema 1.1: Struttura schematica del processo di creazione dei feature vector

1.1 Quantum Espresso

Quantum Espresso [1, 2, 3] è il motore della dinamica molecolare, esso si occupa di utilizzare i principi della meccanica quantistica per produrre i frame che descrivono la dinamica molecolare per il materiale in esame.

1.2 SOAP

In questa sezione ci poniamo come obiettivo quello di dare una breve ma decente spiegazione, per quanto possibile completa e chiusa, del metodo SOAP (Smooth Overlap of Atomic Positions). Come indicato sopra questa procedura è implementata tramite DDescribe.

Il punto fondamentale per comprendere la necessità di utilizzare una procedura come SOAP risiede nel concetto di invarianze, è infatti comune per le leggi della fisica essere invarianti per determinate proprietà, come lo possono essere le rotazioni, le traslazioni spaziali e lo scambio tra atomi uguali. Queste simmetrie emergono in modo naturale dalle leggi utilizzate, ma come un fisico deve impararle e con fatica ricavarle, anche un modello di ML necessiterebbe di imparare queste simmetrie ogni singola volta, questo richiede una spesa sia dal punto di vista computazionale che da quello temporale. Questo problema viene facilmente risolto utilizzando una rappresentazione che sia naturale espressione delle

simmetrie del sistema, in questo modo non sarà necessario il training sulle simmetrie, rendendo il tutto più semplice.

Generazione della mappa di densità

Il primo step è quello di costruire la densità atomica associata ad un determinato Z tramite una funzione gaussiana. Questo ci permette di passare da una forma discreta ad una continua. Ottenibile applicando la seguente:

$$\rho_Z(\vec{r}) = \sum_{\alpha}^Z e^{-\frac{|\vec{r}-\vec{r}_{\alpha}|^2}{2\sigma^2}} \quad (1.1)$$

quello che stiamo facendo è considerare un atomo con Z fissato qualunque e lo consideriamo come l'origine di un sistema di coordinate e per ogni singolo atomo α con lo stesso numero Z costruiamo una gaussiana. Nella formulazione (1.1) r_{α} indica la distanza tra il centro dell'atomo α vicino e quello centrato con il sistema di riferimento definito sopra. Questa possiede quindi un picco in presenza dell'atomo. Dato che σ viene utilizzato per definire lo zoom con cui analizziamo il sistema, minore è il valore di σ più il risultato sarà perturbato anche da minime modifiche alla struttura, come una leggera modifica alla lunghezza dei legami (e conseguentemente della posizione relativa).

Questo step viene ripetuto per ogni singolo atomo, ottenendo come risultato che ogni atomo ha una mappa di densità dei suoi vicini, associata ad esso.

Il fine ultimo della procedura descritta è quello di slegarci dal sistema di coordinate, usando infatti solo informazioni riguardanti la posizione relativa, rendiamo il tutto invariante per traslazioni. La somma su tutti gli atomi con il medesimo Z rende inoltre invariante la rappresentazione rispetto allo scambio di due atomi uguali. La nuvola che viene così creata non è invariante per rotazioni in quanto, se il centro del sistema di riferimento temporaneo menzionato sopra non è simmetrico per rotazione, allora non lo sarà neanche la nuvola risultate.

Nel caso siano presenti elementi con diverso numero atomico, come non di rado succede, la mappa di densità viene separata in base ad esso, quindi nel caso avessimo Z_1 e Z_2 allora avremmo due mappe distinte, ρ_{Z_1} e ρ_{Z_2} . Questo mantiene l'invarianza per permutazione e permette anche di distinguere le diverse specie chimiche presenti.

Vogliamo dare adesso una rappresentazione grafica di questo concetto, facciamo riferimento alla seguente immagine 1.1:

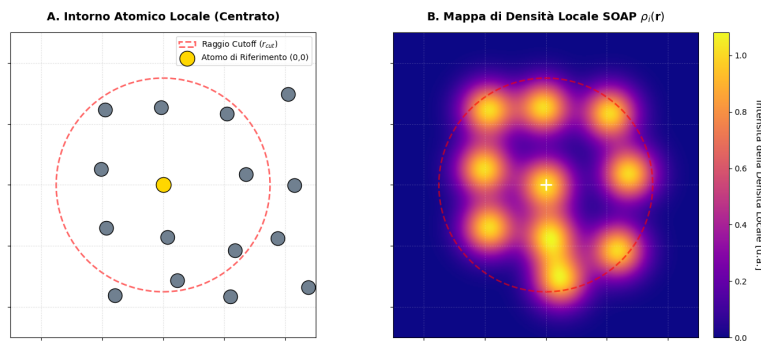


Figura 1.1: Rappresentazione generata tramite matplotlib del primo step della procedura SOAP

questa immagine è la somma di tutte le singole mappe prodotte per gli atomi interni al r_{cut} , dove è stato preso come riferimento l'atomo color oro posto al centro del sistema di coordinate. Il risultato finale si ottiene prendendo come atomo color oro, a turno, ognuno degli atomi presenti. Per semplicità di rappresentazione la struttura atomica è stata generata aggiungendo rumore ad un cristallo periodico, questo non rappresenta nessuna struttura particolare, è solo una semplice griglia sulle cui intersezioni è stato posizionato un generico atomo.

Il controllo che viene dato dal parametro σ della gaussiana è rappresentato nella seguente 1.2:

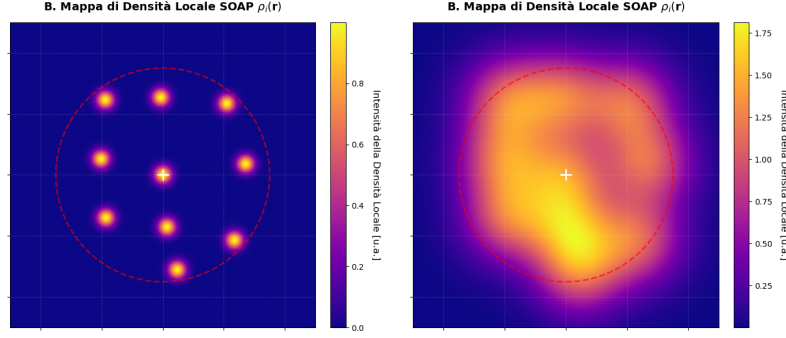


Figura 1.2: Rappresentazione generata tramite matplotlib del primo step della procedura SOAP per diversi valori del parametro σ

dove il pannello a sx rappresenta il caso di una σ eccessivamente piccola e quello a dx il caso di una σ eccessivamente grande. In entrambi i casi è possibile notare come l'informazione che se ne ricava non è adeguata agli scopi descritti sopra. Infatti come detto vogliamo che l'aggiunta di una piccola quantità di rumore non modifichi in modo sensibile il fattore di somiglianza tra le due strutture. Nel primo caso la minima variazione di posizione porta a decretare che gli ambienti sono sensibilmente diversi, mentre nel secondo caso, anche una decisa modifica delle posizioni, porta alla conclusione che i due ambienti siano simili.

Per quanto riguarda il valore di r_{cut} , esso controlla il significato che diamo ad "atomi vicini". Se fosse troppo piccolo si perdono informazioni riguardanti la struttura in esame, mentre se troppo grande arriviamo ad avere un vettore di feature complesso e pesante che non fornisce particolari vantaggi.

Espansione in funzioni di base

A questo punto ci poniamo di trovare un metodo che ci permetta di riassumere in modo compatto una funzione continua come quella rappresentata dalla mappa di densità. Per fare questo ci procuriamo delle funzioni di base, nel nostro caso radiali e angolari.

L'espansione in serie per la mappa di densità è data da:

$$\rho_Z(\vec{r}) = \sum_{n=1}^{n_{max}} \sum_{l=1}^{l_{max}} \sum_{m=-l}^l c_Z(;n,l,m) g(r;n) Y(\theta,\phi;l,m) \quad (1.2)$$

dove il ; è posto a separatore tra le variabili e i parametri da cui dipendono le funzioni. Il coefficiente c_Z è per la relativa specie in considerazione, come indicato sopra in presenza di specie diverse si costruiscono mappe separate.

In particolare si ha che:

$$c_Z(;n,l,m) = \int_{\mathbb{R}^3} dr d\theta d\phi g(r;n) Y(\theta,\phi;l,m) \rho_Z(\vec{r}) \quad (1.3)$$

dove le $g(r;n)$ sono funzioni radiali e $Y(\theta,\phi;l,m)$ sono armoniche sferiche. Quello che stiamo andando a determinare tramite il coefficiente c_Z è la "misura" della nostra mappa di densità lungo una determinata componente di base, in questo caso funzioni radiali e armoniche sferiche (si utilizzano queste funzioni per le loro proprietà matematiche). Possiamo quindi adesso utilizzare questi coefficienti (un semplice vettore numerico) per ricostruire un oggetto tridimensionale come lo è la mappa di densità.

In questa sezione non ci dilunghiamo sui dettagli della forma di queste funzioni.

Lo scopo di questo step è quello di semplificare la rappresentazione, come detto passiamo da una struttura tridimensionale complessa ad un singolo vettore numerico. Non abbiamo ancora aggiustato le restanti invarianze, in quanto alla fine di questo step manca ancora l'invarianza per rotazione, la sua risoluzione sarà il cardine dello step successivo.

Calcolo del power spectrum

Per rendere la nostra rappresentazione invariante per rotazioni andiamo a calcolare il power spectrum parziale, ottenuto dalla seguente equazione:

$$p_{Z_1 Z_2}(n, n', l) = \pi \sqrt{\frac{8}{2l+1}} \sum_{m=-l}^l \overline{c_{Z_1}(n, l, m)} c_{Z_2}(n', l, m) \quad (1.4)$$

dove il primo coefficiente è sotto complessa coniugazione, nel caso si usino solo armoniche sferiche reali, essa diventa superflua. La presenza di Z_1 e Z_2 indica che questo viene calcolato per ogni coppia unica (indipendentemente dall'ordine) di specie chimiche presenti. Nel contesto del dataset A, quello che si produce è (Z_{Ta}, Z_{Ta}) , (Z_O, Z_O) e (Z_{Ta}, Z_O) .

Senza entrare nei dettagli, diciamo che questo processo rende il risultato invariante dal punto di vista delle rotazioni, in quanto, le rotazioni possono essere rappresentate tramite matrici unitarie (oppure ortogonali nel caso reale), il processo descritto sopra "trasforma" la matrice di rotazione (con cui possiamo rappresentare la rotazione della mappa di densità rispetto ad un qualunque sistema di riferimento) nella matrice identità, che quindi non modifica il valore del coefficiente.

Le correlazioni che si trovano per valori di Z_1 e Z_2 diversi tra loro servono per tenere traccia delle posizioni relative delle diverse specie atomiche.

Risultato finale

Dopo aver calcolato tutti i vari elementi del power spectrum, si strutturano come un vettore, la cui dimensione viene determinata dalla seguente formula:

$$\text{dimensione} = (l_{max} + 1) \frac{1}{2} (S \cdot n_{max})(S \cdot n_{max} + 1) \quad (1.5)$$

Dove S rappresenta il numero totale di specie diverse presenti nel materiale in analisi, n_{max} e l_{max} sono gli stessi parametri precedentemente definiti.

Non è immediato capire come siamo arrivati alla seguente formula, conseguentemente diamo di seguito una spiegazione. Per prima cosa consideriamo il caso in cui $Z_1 = Z_2$, il più semplice e che usiamo come punto di partenza per la successiva generalizzazione.

Ci serve mantenere come riferimento (1.4). In questa reference se si ha un solo Z allora il numero finale di $p_{Z_1 Z_2}(n, n', l)$ è dato solo da n , n' e l . Per quanto riguarda l ci sono esattamente $l_{max} + 1$ valori disponibili (in quanto si parte da zero), per capire come comportarsi con gli n , utilizziamo un piccolo aiuto grafico:

	n_1	n_2	$\dots$	n_m
n'_1	11	12	$\dots$	1m
n'_2	21	22	$\dots$	2m
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
n'_m	m1	m2	$\dots$	mm

Schema 1.2: Struttura schematica per la combinazione dei valori di n e n' nel power spectrum

dato che nel power spectrum gli n compaiono in coefficienti che definiscono l'informazione strutturale, con Z diversi, fanno riverimento a mappe di densità diverse. Inoltre ricordiamo che n indica una sorta di distanza radiale. Questo significa che per il medesimo Z , la combinazione di (n, n') produce lo stesso risultato di (n', n) in quanto la mappa di densità è la medesima. Per diminuire la dimensione del vettore finale consideriamo solo gli elementi unici, ad esempio la parte triangolare superiore, ma allora il numero di elementi è la somma di n_m elementi sulla prima riga, $n_m - 1$ sulla seconda e così via fino ad arrivare ad un singolo elemento. Questo coincide con la somma dei primi n_m numeri naturali, risolta da Gauss come:

$$\text{somma}(a) = \frac{a(a+1)}{2} \quad (1.6)$$

Consideriamo adesso la presenza di un numero di specie maggiore di 1, allora dobbiamo considerare il fatto che lo scambio degli n produce risultati diversi, in quanto fa riferimento a mappe di densità diverse. Le combinazioni di Z (che sono simmetriche per lo scambio in quanto lo è il prodotto) si possono immaginare con uno schema identico a quello 1.2 dove, al posto di n mettiamo i vari Z , da questo ricaviamo la seguente distinzione:

- (Z_i, Z_i) gli elementi sulla diagonale, sono esattamente pari al numero di specie chimiche distinte presenti, ognuno di essi ha una combinazione di n pari al numero definito in precedenza (1.6)
- (Z_i, Z_j) con $i \neq j$, questi per la simmetria già menzionata, sono esattamente gli elementi sul triangolo superiore meno quelli sulla diagonale. Possiamo facilmente vedere che questa somma di ottiene con la formula di Gauss (1.6), dove adesso $a = S - 1$, con S numero di specie presenti. Data la perdita di simmetria ognuna di queste coppie ha n^2 elementi distinti da tenere in considerazione

Se procediamo a mettere tutto assieme otteniamo la seguente espressione:

$$\text{dimensione} = (l_{max} + 1) \left(\frac{S \cdot n_m(n_m + 1)}{2} + \frac{S(S - 1)n_m^2}{2} \right) \quad (1.7)$$

che se procediamo a semplificare otteniamo l'espressione indicata in (1.5).

In conclusione, possiamo rappresentare l'insieme di questi coefficienti come un vettore, la cui dimensione è fissata e non dipende dall'input (quanti vicini / come è strutturato l'ambiente per un dato atomo) ma solo dai parametri che abbiamo utilizzato per descriverlo.

Schema riassuntivo

Diamo adesso una rappresentazione riassuntiva del processo completo, compresa anche la possibilità di dinamica molecolare suddivisa per frames. In particolare, il metodo di archiviazione della struttura di SOAP e la sua indicizzazione / organizzazione è lasciato pienamente in gestione a DScript, notiamo che l'ordine del vettore di feature non è trascurabile.

1.3 Potenziali GAP

Lo step successivo alla descrizione di un ambiente atomico tramite SOAP è quello di generare un potenziale che sia poi utilizzabile per fare previsioni di dinamica molecolare. Il metodo che abbiamo deciso di utilizzare in questa analisi è quello dei potenziali GAP (Gaussian Approximation Potentials), questa tecnica si basa sul concetto della GPR (Gaussian Process Regression).

Il core concept di questo metodo è quello di scomporre l'energia totale del sistema nella somma dei singoli contributi atomici. Come per SOAP vogliamo utilizzare una descrizione locale per poter ottenere una complete picture del sistema in esame.

1.4 Addestramento

Per la fase di training andremo ad utilizzare dati generati da Quantum Espresso e un potenziale GAP. Partiamo esponendo in breve la struttura del processo che andremo ad utilizzare.

La funzione svolta dal potenziale GAP è quella di associare determinate energie e forze, fornite come detto da Quantum Espresso, ad una specifica configurazione geometrica, questa descritta tramite SOAP.

Questa associazione viene poi utilizzata per indovinare il valore di energia di un atomo dato il suo intorno geometrico, in particolare, preso un atomo X ne viene calcolato il SOAP, questo, tramite una funzione di kernel, viene confrontato con gli ambienti della fase di training ed infine viene estratto un valore probabile per l'energia. Data la struttura del metodo, un punto che vogliamo sottolineare è che l'accuratezza decresce velocemente fuori da scenari compatibili con i dati di training, è ovviamente possibile arginare se non risolvere il problema aggiungendo diverse tecnologie al metodo, queste non sono però esaminate in questa trattazione.

1.5 Simulazione

Per simulare una dinamica molecolare usiamo un processo iterativo, partiamo da una configurazione che facciamo evolvere nel tempo in base alle forze calcolate tramite l'energia ricavata a partire dai potenziali GAP, questo risultato viene inserito nuovamente all'inizio del ciclo. Si ottiene in questo modo una traiettoria di dinamica molecolare per il sistema in esame.

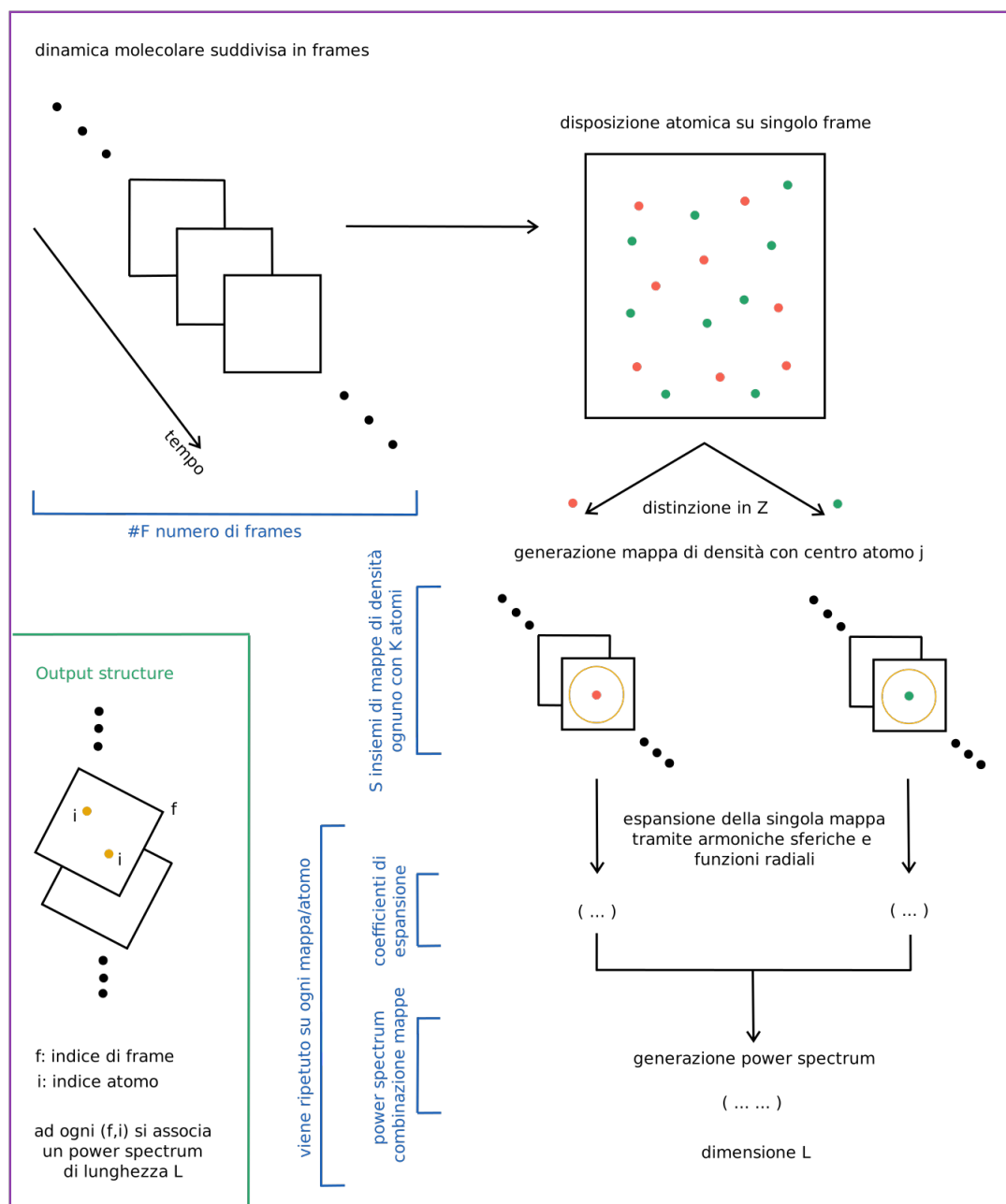


Figura 1.3: Rappresentazione schematica della totalità del processo SOAP

Capitolo 2

Discussione teorica sulle librerie python utilizzate

2.1 ASE

Questa libreria [4] ci permette di analizzare le simulazioni di dinamica molecolare prodotte tramite QE. Essenzialmente possiamo dire che essa svolge il ruolo di ponte tra una struttura fatta di coordinate e una che sia computazionalmente utilizzabile. Questo ci permette di trasportare le informazioni che QE utilizza per svolgere i calcoli, come le condizioni di periodicità, in un formato poi accessibile anche da altre librerie. Inoltre questa libreria ci permette di svolgere semplici simulazioni di dinamica molecolare.

2.1.1 Lettura dei file di Quantum Espresso

In particolare per QE abbiamo utilizzato i seguenti:

Listing 2.1: Snippet usato per il dataset A

```
1 frame = ase.io.read(file_name, index=":", format="espresso-out")
```

dove `index=":"` viene utilizzato per leggere tutti i frame presenti nel file di input e `format="espresso-out"` serve per impostare la tipologia di file da leggere.¹

La funzione `read` ci restituisce un oggetto di tipo `atoms`², che costituisce l'oggetto python che contiene le informazioni, le quali, definiscono il materiale da simulare, estratte dai file di output di QE.

2.1.2 Simulazione con un potenziale ML

I potenziali generati tramite ML vengo utilizzati come dei *calculator*³ per la dinamica molecolare. Questo oggetto può essere descritto come una calcolatrice che data una configurazione restituisce i valori delle energie e delle forze.

Lo step successivo è utilizzare un motore di dinamica molecolare⁴ per muovere gli atomi ed ottenere il frame successivo.

Diamo adesso una descrizione del codice utilizzato per svolgere questo processo, notiamo che il potenziale addestrato tramite QUIP, deve essere caricato tramite la libreria quippy, essendo necessario per la discussione sulla dinamica molecolare, lo riportiamo in questa sezione.

¹La specifica sezione della documentazione è reperibile al seguente <https://ase-lib.org/ase/io/io.html>

²La specifica sezione della documentazione è reperibile al seguente <https://ase-lib.org/ase/atoms.html>

³Si veda la documentazione: <https://docs.ase-lib.org/ase/calculators/calculators.html>

⁴Si veda la documentazione: <https://docs.ase-lib.org/ase/md.html>

Listing 2.2: Loading delle librerie

```

1  from ase.md.langevin import Langevin
2  from ase.md.velocitydistribution import
   MaxwellBoltzmannDistribution
3  from ase import units
4
5  # Importiamo il calcolatore Quippy per leggere il potenziale GAP
6  from quippy.potential import Potential

```

dove `Langevin` è il motore di dinamica molecolare che ci permette di mantenere la temperatura costante, `MaxwellBoltzmannDistribution` ci permette di assegnare una velocità iniziale agli atomi da simulare.

Il passo successivo prevede di caricare il potenziale di ML e di trasformarlo in un calcolatore:

Listing 2.3: Loading del potenziale

```

1  calc = Potential('IP GAP', param_filename=file_potenziale)
2  calc.name = "GAP"

```

Come step iniziale della simulazione carichiamo il primo frame dei dati di training e assegnamo a questo oggetto `atomi` il calcolatore definito allo step precedente

Listing 2.4: Loading del frame iniziale e assegnazione del calcolatore

```

1  atomi = read(file_iniziale, index=0)
2  atomi.set_calculator(calc)

```

Possiamo adesso procedere alla costruzione del motore della dinamica molecolare

Listing 2.5: Costruzione del motore per la dinamica molecolare

```

1  temperatura_K = #setting per la temperatura in kelvin da mantenere
   durante la dinamica
2  MaxwellBoltzmannDistribution(atomi, temperature_K=temperatura_K) #
   assegna le velocità iniziali
3  timestep = * units.fs #l'intervallo temporale tra uno step ed il
   successivo
4
5  dyn = Langevin(
6      atomi,
7      timestep,
8      temperature_K=temperatura_K,
9      friction= / units.fs, # Coefficiente di attrito per il
   termostato
10     trajectory='.traj' # Il file dove salveremo i risultati
11 )

```

in particolare dove troviamo `units.fs`, lo scopo è quello di convertire in automatico i valori *human frinedly* nelle unità che utilizza ASE. Lo scopo del parametro `frinction` è quello di definire l'attrito del bagno termico, questo serve per mantenere la temperatura il più possibile costante. Il motore utilizzato è `Langevin`, questo ci permette di mantenere il valore di temperatura costante, come per le simulazioni di Quantum Espresso.

Lo step finale precede di avviare la simulazione e farla correre per un determinato numero di iterazioni

Listing 2.6: Running della simulazione

```

1  dyn.run(passi_totali)

```

2.2 DScribe

Questa è la libreria [5] che si occupa di trasformare la struttura atomica, definita tramite ASE, in un qualcosa di comprensibile dai modelli di ML. Lo scopo principale di questa procedura è quello di rendere il training di un modello il più semplice possibile. L'idea è quella di creare un oggetto che identifichi in modo univoco una determinata struttura atomica e che sia invariante per modifiche "matematiche" della struttura stessa, come possono esserlo le rotazioni; se così non fosse il modello dipenderebbe da un sistema di coordinate e andrebbe sottoposto a training anche per le diverse rotazioni. Questo processo porta alla produzione di un descriptor, o vettore di feature, che contiene l'informazione sulle invarianze delle leggi fisiche che descrivono il sistema. La prima cosa che facciamo notare è che separando il descriptor dalla fase di training è possibile riutilizzarlo con un diverso modello, diminuendo così il tempo necessario per l'addestramento. Abbiamo deciso di utilizzare il descriptor SOAP, per la rappresentazione locale della struttura atomica in analisi. [6, 7]

Riportiamo adesso l'uso che abbiamo fatto di questo pacchetto:

Listing 2.7: Snippet usato per la creazione delle feature tramite SOAP

```

1  soap_obj = SOAP(
2      species= , # tutti gli atomi nella struttura in esame
3      periodic= , # condizione di periodicità
4      r_cut= , # raggio di cutoff
5      n_max= , # numero funzioni radiali
6      l_max= # grado massimo armoniche sferiche
7  )
8
9  features = soap_obj.create(QE_data, n_jobs=-1)
```

dove `soap_obj` è interpretabile come una configurazione per il SOAP, `species` definisce tutti gli atomi che saranno incontrati nella struttura in esame, `periodic` serve per impostare l'uso della struttura periodica (precedentemente definita in ASE), `r_cut` definisce un cutoff per la rappresentazione locale (espresso in Å), `n_max` è il numero di funzioni radiali di base, `l_max` il massimo grado delle armoniche sferiche⁵. Il valore di σ discusso in (1.1) assume come valore predefinito 1.

In particolare l'accuratezza della rappresentazione, come il numero delle feature, cresce proporzionalmente al valore di `n_max` e `l_max`.

La funzione `create` si occupa poi di creare il vero e proprio SOAP, in input viene passato `QE_data`, ovvero la struttura atomica in analisi definita tramite la libreria ASE.

Il risultato di questi calcoli è un complesso oggetto, indicizzato come segue:

```
[num frames, num atomi, feature vector]
```

Note sulla struttura matematica

DScribe, in particolare, per le armoniche sferiche usa la loro versione puramente reale, in modo da evitare l'utilizzo dell'algebra complessa e come funzioni radiali delle GTO (gaussian type orbitals) in modo da rendere il tutto computazionalmente veloce e accurato.

2.3 Quippy

Questa è la libreria che si occupa della gestione del potenziale GAP [8, 9].

In particolare andremo ad utilizzare la funzione `gap_fit`⁶

⁵La documentazione è reperibile al seguente: <https://singroup.github.io/dscribe/latest/tutorials/descriptors/soap.html>

⁶La documentazione è reperibile al seguente: https://libatoms.github.io/GAP/gap_fit.html

Listing 2.8: Snippet per il calcolo del GAP

```
1
2 gap_fit
3 at_file=                               #nome file.xyz con i dati di training
4 default_sigma={                         #definisce le incertezze sui dati:
5     -                                   #incertezza sull'energia
6     -                                   #incertezza sulle forze
7     -                                   #non presente nei dati
8     -                                   #non presente nei dati
9 }
10 gap={soap                              #tipologia di descrittore, nel
11     # nostro caso SOAP
12     l_max=                             #numero funzioni radiali
13     n_max=                             #grado massimo armoniche sferiche
14     atom_sigma=                        #"zoom" della mappa di densità
15     zeta=                              #esponente per il kernel
16     cutoff=                            #raggio di cutoff
17     covariance_type=                   #tipologia di kernel per il
18     # confronto degli ambienti
19     n_sparse=                           #numero punti per lo sparse method
20     sparse_method=                      #tipologia di sparse method
21     delta=                              #?
22 }
23 e0=                                    #energia di atomo libero
24 energy_parameter_name=energy            #config name per file input
25 force_parameter_name=forces             #config name per file input
26 gp_file=                               #nome file.xml di output
```

dopo aver generato il file del potenziale, esso può essere utilizzato come `calculator` per la dinamica molecolare tramite ASE.

Capitolo 3

Test: dataset A

Questo dataset consiste di una simulazione di MD eseguita con QE per una struttura cristallina costituita da Ta_2O_5 , pentossido di ditantalio (a cui ci riferiamo anche come tantala).

Esso è composto da una serie di file di QE che costituiscono la dinamica molecolare di una struttura di tantala a circa 3000K.

Usando la libreria ASE abbiamo generato una rappresentazione della cella atomica dai dati di QE analizzati, essendo una simulazione di dinamica molecolare, riportiamo solo uno degli step, a cui diamo il nome di frame. Riportiamo di seguito questa immagine, disegnata tramite matplotlib 3.1.

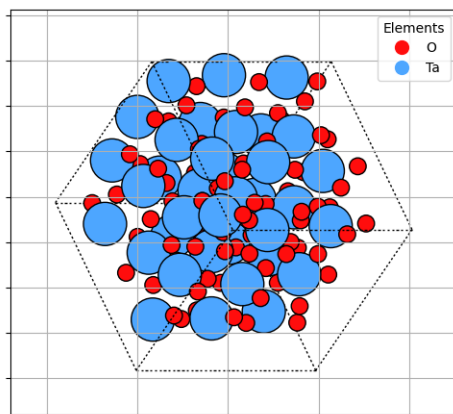


Figura 3.1: Rappresentazione generata tramite ASE della struttura atomica del dataset A

3.1 SOAP con DScibe

Considerando la discussione svolta per spiegare SOAP unita alla spiegazione riguardante DScibe, vogliamo dare un esempio pratico di utilizzo del metodo considerando un contesto sul quale conosciamo già il risultato.

Dove abbiamo utilizzato Listing 2.7 nel seguente modo

Listing 3.1: Snippet usato per il dataset A

```
1 soap_obj = SOAP(  
2     species=["Ta", "O"],  
3     periodic=True,  
4     r_cut=5.0,  
5     n_max=8,  
6     l_max=6  
7 )
```

```

8
9 features = soap_obj.create(QE_data, n_jobs=-1)

```

Nota sui parametri

Dato che ci stiamo ponendo come scopo quello di verificare che il metodo sia utilizzabile, i parametri riportati sopra non sono stati in alcun modo ottimizzati, essi sono stati reperiti negli esempi della documentazione grazie a strumenti di AI che ne hanno fatto la revisione. Data la spiegazione che abbiamo dato per essi, il test che si potrebbe svolgere per la loro ottimizzazione è un tradeoff tra velocità di calcolo e accuratezza dei risultati rispetto a quelli prodotti da Quantum Espresso.

Consideriamo l'atomo 42 e calcoliamo un *kernel* tra i frame successivi, questo significa che seguiamo un singolo atomo lungo tutta la dinamica molecolare.

Un kernel è una funzione che ci permette di misurare quanto due descrittori soap (il power spectrum) siano simili, e conseguentemente gli ambienti da cui derivano. Nel nostro caso usiamo il kernel più semplice possibile:

$$\text{Kernel}(\vec{p}_1, \vec{p}_2) = \frac{\vec{p}_1 \cdot \vec{p}_2}{|\vec{p}_1| |\vec{p}_2|} \quad (3.1)$$

Il risultato che ci aspettiamo è che lungo tutti i frame il valore di questa funzione sia circa uno, data la struttura dei dati, infatti ogni frame rappresenta un piccolo scostamento temporale di una dinamica molecolare continua.

Se produciamo un grafico dove lungo l'asse x poniamo un indice (1 rappresenta il confronto tra il frame 1 e 2, e così per tutti gli altri indici) e sull'asse delle y mettiamo il valore del kernel otteniamo:

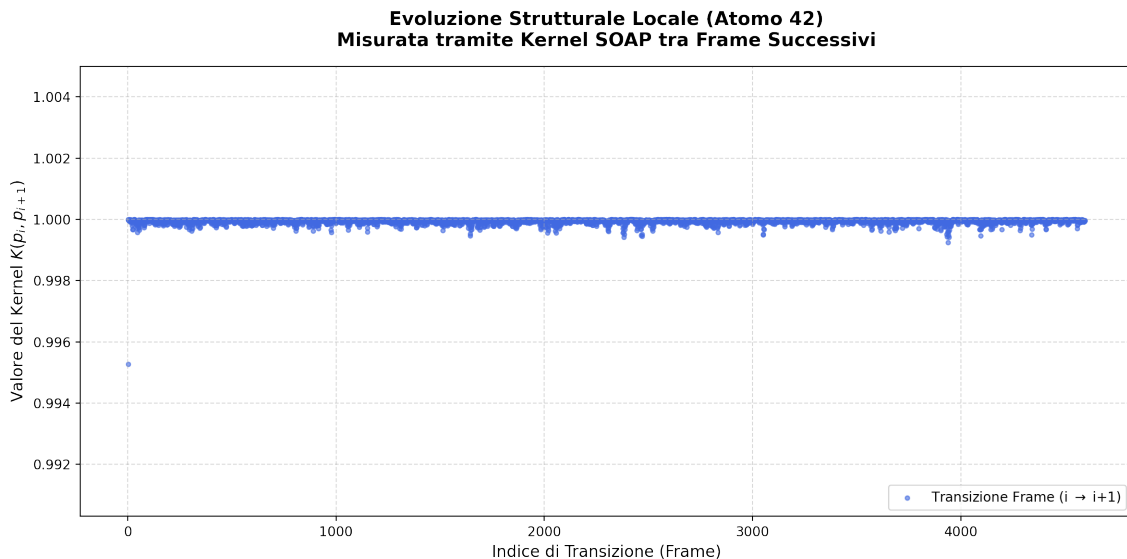


Figura 3.2: Rappresentazione del kernel per i frame del Dataset A

Il risultato è esattamente quello atteso, l'unico punto che si discosta è il primo, non riteniamo questo punto particolarmente problematico.

3.2 Prima sessione di testing

In questa sezione spiegheremo come è stato utilizzato il dataset A per sperimentare il funzionamento del workflow descritto all'inizio del documento.

3.2.1 Addestramento potenziale con Quippy

Listing 3.2: Snippet usato per il calcolo del GAP per il dataset A

```

1
2 gap_fit
3 at_file=train_data.extxyz
4 default_sigma={0.0002 0.1 0.0 0.0}
5 gap={soap
6     l_max=2
7     n_max=2
8     atom_sigma=0.5
9     zeta=4
10    cutoff=5.0
11    covariance_type=dot_product
12    n_sparse=500
13    sparse_method=cur_points
14    delta=1
15    }
16 e0={Ta:-9884.889316093:0:-561.82449957}
17 energy_parameter_name=energy
18 force_parameter_name=forces
19 gp_file=out_potenziale_gap-500-n2-12.xml
20 > potenziale_quippy-500-n2-12.log

```

Nota sui parametri

Dato che ci stiamo ponendo come scopo quello di verificare che il metodo sia utilizzabile, i parametri riportati sopra non sono stati in alcun modo ottimizzati, essi sono stati reperiti negli esempi della documentazione grazie a strumenti di AI che ne hanno fatto la revisione. Data la spiegazione che abbiamo dato per essi, il test che si potrebbe svolgere per la loro ottimizzazione è un tradeoff tra velocità di calcolo e accuratezza dei risultati rispetto a quelli prodotti da Quantum Espresso.

Per l'errore relativo alle energie, abbiamo usato lo stesso ordine di grandezza che quantum espresso utilizzava per decretare la convergenza della simulazione. Le energie per gli atomi isolati `e0` sono state determinate tramite una simulazione di QE. Il grado in `n` e `l` per le armoniche sferiche sono stati impostati come tali per la limitatezza della RAM disponibile sul sistema usato per il training, i problemi relativi all'accuratezza dei dati, come si può vedere sotto, in prima analisi possono essere imputabili a questa brusca approssimazione.

La nomenclatura dei parametri è standard e deriva dall'aver prodotto il file con ASE.

Gli altri parametri, come detto sono stati forniti dalla documentazione e andrebbero ottimizzati. Non escludiamo che possano esserci problemi di accuratezza dovuti alla scelta di questi valori.

3.2.2 Verifica delle energie

Per verificare se il potenziale così addestrato sia in grado di predire in modo corretto le energie abbiamo estratto le configurazioni geometriche prodotte da Quantum Espresso e utilizzato il potenziale addestrato per predire l'energia, successivamente abbiamo confrontato i valori predetti con quelli calcolati da Quantum Espresso.

Training set

Per prima cosa verifichiamo se utilizzando come dati di input quelli usati durante la fase di addestramento otteniamo il risultato atteso, se così non fosse potremmo velocemente concludere che l'addestramento non è andato a buon fine.

Dal grafico sopra possiamo vedere che il risultato è compatibile con le aspettative, e le energie predette assomigliano a quelle prodotte da Quantum Espresso.

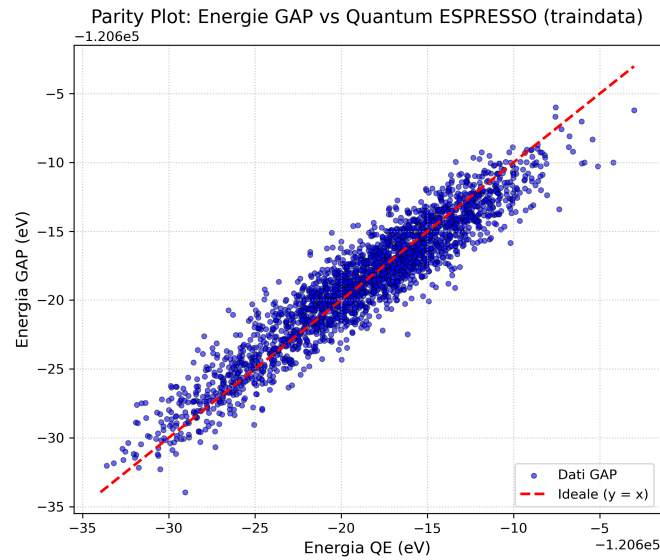


Figura 3.3: Confronto tra i valori di energia potenziale predetti dal potenziale GAP e quelli calcolati tramite Quantum Espresso

Test set

In questa fase invece vogliamo verificare come si comporta il potenziale nel caso di dati su cui non è stato addestrato, ricordiamo che questo set è composto dalla seconda metà della dinamica totale descritta dalla totalità del dataset A.

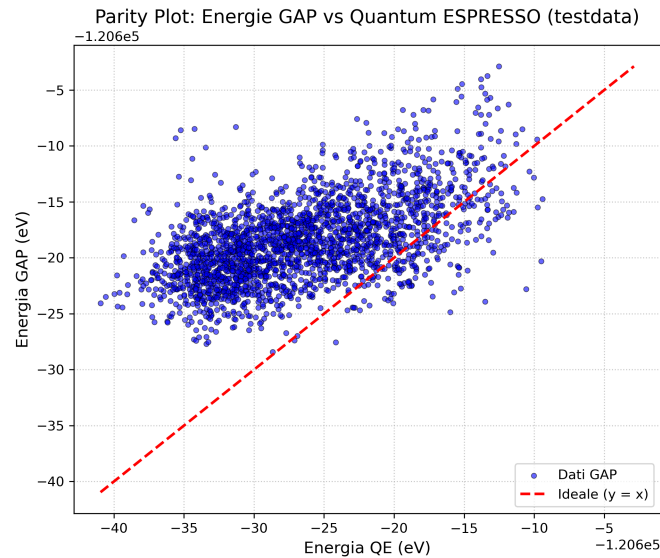


Figura 3.4: Confronto tra i valori di energia potenziale predetti dal potenziale GAP e quelli calcolati tramite Quantum Espresso

Possiamo vedere che i risultati non si allineano perfettamente con le aspettative, ma date le limitazioni dei parametri di addestramento del potenziale e dalla rapidità di calcolo ottenuta, possiamo considerare questo un buon risultato iniziale.

Bibliografia

- [1] P. Giannozzi, S. Baroni, N. Bonini, M. Calandra, R. Car, C. Cavazzoni, D. Ceresoli, G. L. Chiarotti, M. Cococcioni, I. Dabo, A. Dal Corso, S. Fabris, G. Fratesi, S. de Gironcoli, R. Gebauer, U. Gerstmann, C. Gougoussis, A. Kokalj, M. Lazzeri, L. Martin-Samos, N. Marzari, F. Mauri, R. Mazzarello, S. Paolini, A. Pasquarello, L. Paulatto, C. Sbraccia, S. Scandolo, G. Sclauzero, A. P. Seitsonen, A. Smogunov, P. Umari e R. M. Wentzcovitch. “QUANTUM ESPRESSO: a modular and open-source software project for quantum simulations of materials”. In: *Journal of Physics: Condensed Matter* 21.39 (2009). DOI: 10.1088/0953-8984/21/39/395502.
- [2] P. Giannozzi, O. Andreussi, T. Brumme, O. Bunau, M. Buongiorno Nardelli, M. Calandra, R. Car, C. Cavazzoni, D. Ceresoli, M. Cococcioni, N. Colonna, I. Carnimeo, A. Dal Corso, S. de Gironcoli, P. Delugas, R. A. DiStasio Jr, A. Ferretti, A. Floris, G. Fratesi, G. Fugallo, R. Gebauer, U. Gerstmann, F. Giustino, T. Gorni, J. Jia, M. Kawamura, H.-Y. Ko, A. Kokalj, E. Küçükbenli, M. Lazzeri, M. Marsili, N. Marzari, F. Mauri, N. L. Nguyen, H.-V. Nguyen, A. Otero-de-la-Roza, L. Paulatto, S. Poncé, D. Rocca, R. Sabatini, B. Santra, M. Schlipf, A. P. Seitsonen, A. Smogunov, I. Timrov, T. Thonhauser, P. Umari, N. Vast, X. Wu e S. Baroni. “Advanced capabilities for materials modelling with Quantum ESPRESSO”. In: *Journal of Physics: Condensed Matter* 29.46 (2017). DOI: 10.1088/1361-648X/aa8f79.
- [3] P. Giannozzi, O. Baseggio, P. Bonfà, D. Brunato, R. Car, I. Carnimeo, C. Cavazzoni, S. de Gironcoli, P. Delugas, F. Ferrari Ruffino, A. Ferretti, N. Marzari, I. Timrov, A. Urru e S. Baroni. “Quantum ESPRESSO toward the exascale”. In: *The Journal of Chemical Physics* 152.15 (2020). DOI: <https://doi.org/10.1063/5.0005082>.
- [4] *ASE documentation*. URL: <https://ase-lib.org/>.
- [5] *DShcribe documentation*. URL: <https://singroup.github.io/dscribe/latest/>.
- [6] Lauri Himanen, Marc O.J. Jäger, Eiaki V. Morooka, Filippo Federici Canova, Yashasvi S. Rana-wat, David Z. Gao, Patrick Rinke e Adam S. Foster. “DShcribe: Library of descriptors for machine learning in materials science”. In: (2019).
- [7] Jarno Laakso, Lauri Himanen, Henrietta Homm, Eiaki V. Morooka, Marc O.J. Jäger, Milica Todo-rovic e Patrick Rinke. “Updates to the DShcribe library: New descriptors and derivatives”. In: *The Journal of Chemical Physics* 158.23 (2023).
- [8] *QUIP and quippy documentation*. URL: <https://libatoms.github.io/QUIP/>.
- [9] *QUIP and quippy documentation - GAP*. URL: <https://libatoms.github.io/GAP/>.

-